ORBITWATCH PLATFORM

Space Domain Awareness, Solar Weather Telemetry & Atmospheric Drag Analytics
Comprehensive Architectural & Engineering Deep Analysis

Project Version: Production 1.0 (Phases 1-10)	**Primary Platform:** Windows 11 / Linux (Docker Engine)
Author / Engineer: Param Patil	**Generated:** 2026-10-14 17:06 UTC
Repository: ParamPatil-03/Orbitwatch_DE_Miniproject	**Status:** Fully Verified (43 Automated Tests Passing)

1. Executive Summary & Problem Statement

The rapid proliferation of commercial satellite mega-constellations (e.g., Starlink, OneWeb) and historical fragmentation debris has created an acute congestion crisis in Low Earth Orbit (LEO). Predicting satellite collision avoidance and atmospheric re-entry requires modeling the interaction between orbital mechanics and upper atmospheric drag. Atmospheric drag in LEO (200–600 km) is fundamentally driven by solar activity: Extreme Ultraviolet (EUV) radiation expands Earth's thermosphere, multiplying neutral density by up to an order of magnitude and accelerating orbital decay.

ORBITWATCH solves this domain problem by establishing an end-to-end, multi-tier data engineering platform. It ingests continuous satellite ephemerides from **CelesTrak** alongside multi-decade solar radio flux (F10.7) and sunspot indices from **NOAA SWPC**. The platform coordinates polyglot ingestion, event-driven CDC streaming, automated data quality gates, Kimball dimensional star schemas, Lakehouse exports, workflow orchestration, and role-governed executive visualization.

2. Full Architectural Topology & Tech Stack

The platform leverages a polyglot persistence architecture designed for high availability, ACID reliability, and high-throughput analytical scans:

Layer	Component	Technology	Technical Role & Description
Ingestion	REST Microservice	FastAPI + Uvicorn	Async HTTP ingestion of raw external payloads with API auth.
Raw Store	Document Database	MongoDB 7.0	NoSQL store with composite keys for idempotent raw archiving.
Streaming	Event Broker	Redpanda (Kafka API)	Low-latency C++ event broker streaming CDC topics (:19092).
Staging	Relational Staging	PostgreSQL 16	Landing area for raw consumed payloads (staging.*_raw).
Quality	Validation Engine	Great Expectations 1.x	Two-tier validation, dead-letter routing, and HTML docs.
Warehouse	Dimensional Schema	PostgreSQL schema 'dw'	Kimball star schema with surrogate keys and strict FKs.
Lakehouse	Columnar Lake	Delta Lake + DuckDB	ACID Parquet tables in data/lake/ queried via DuckDB.
Orchestration	Workflow DAG	Prefect 3.8	Sequential task execution with retries and failure gates.
Serving	Executive Dashboard	Streamlit 1.65	Role-governed visualizer (dashboard_user) with drag analysis.

3. Ingestion & Event-Driven Streaming Pipeline

Ingestion Layer: External telemetry sources (CelesTrak SATCAT and General Perturbations orbital elements; NOAA SWPC Solar Cycles and Planetary K-indices) are collected via HTTP scrapers. The raw JSON and CSV data streams into a FastAPI service (/ingest/{source}), which writes directly to MongoDB collections: raw_gp, raw_satcat, raw_solar, and raw_k_index. Idempotency is enforced at the MongoDB layer using unique compound indexes on natural composite keys (e.g., norad_cat_id + epoch), guaranteeing zero duplicate writes upon re-scraping.

Event Streaming (CDC): The MongoDB collection states are monitored by a Kafka producer (src/orbitwatch/streaming/producer.py) that converts documents into standardized JSON message envelopes. Each message contains a deterministic UUID5 event ID derived from the natural key. Events are published to Redpanda topics: orbitwatch.gp, orbitwatch.k_index, and orbitwatch.solar_indices. A dedicated consumer (consumer.py) reads the streams, verifies payload integrity, and inserts valid records into PostgreSQL staging tables (staging.*_raw). Malformed payloads or invalid envelopes are intercepted and routed directly into quality.dead_letter.

4. Data Quality Gates, Validation & Governance

The platform implements a stringent two-tier quality gate using **Great Expectations 1.x** before any data is admitted into clean staging or warehouse layers:

- **Tier 1 (Raw Suites):** Evaluates raw consumed records. Validates physical orbital parameter limits: inclination $0 \le i \le 180^\circ$, eccentricity $0 \le e < 1.0$ (parabolic/hyperbolic orbits rejected), mean motion $n > 0$ rev/day, and chronological validity (epoch cannot be in the future relative to UTC execution time).
- **Tier 2 (Clean Suites):** Evaluates relational referential integrity. Ensures valid orbit regime classifications (LEO, MEO, GEO, HEO), verifies that launch date precedes decay date, and flags statistical outliers using Tukey boxplot IQR.
- **Quarantine & Audit Tracking:** Rejected rows are recorded in quality.dead_letter with explicit reason codes (e.g., INCLINATION_OUT_OF_RANGE, FUTURE_EPOCH). Every validation and pipeline execution generates immutable audit trail rows in audit.pipeline_runs and audit.data_lineage.

5. Dimensional Warehouse Architecture (Schema 'dw')

Rather than relying on 3NF normalized tables that require computationally expensive multi-table joins for analytical queries, ORBITWATCH models the warehouse as a Kimball-style Dimensional Star Schema residing in schema dw:

Table Name	Type	Grain / Description	Primary Key	Row Count
dw.dim_date	Dimension	1 row per calendar day (1749 to 2030)	date_key (YYYYMMDD)	102,998
dw.dim_satellite	Dimension	1 row per catalogued satellite (SCD Type 1)	satellite_key (Surrogate)	70,953
dw.dim_orbit_regime	Dimension	LEO, MEO, GEO, HEO, UNKNOWN	regime_id	5
dw.dim_object_type	Dimension	Payload, Rocket Body, Debris, Unknown	object_type_id	4
dw.fact_gp_snapshot	Fact	1 row per satellite per epoch snapshot	snapshot_id	19,366
dw.fact_space_weather_month	Fact	1 row per calendar month (F10.7, Sunspot #)	weather_month_id	3,333
dw.fact_decay_month	Fact	1 row per month per regime (Sum = 35,677)	decay_month_id	1,535

Atomic Truncate-and-Load ETL: scripts/load_warehouse.py executes within a single PostgreSQL ACID transaction. If any foreign key validation or aggregation fails, the entire transaction rolls back automatically, preventing partial corruption. Idempotency was verified across consecutive runs with zero drift in table counts.

6. Lakehouse Layer (Delta Lake & DuckDB)

To offload high-throughput analytical scans from PostgreSQL and support cloud object storage compatibility, ORBITWATCH implements an ACID Lakehouse layer:

- **Delta Lake Export:** scripts/export_lakehouse.py serializes the three fact tables and dim_satellite into Apache Parquet format under data/lake/ wrapped with Delta transaction logs (_delta_log/). Overwrites maintain transaction history and increment version commits (v0 $\rightarrow$ v1).
- **DuckDB Vectorized Engine:** scripts/query_lake.py initializes DuckDB in-process and registers Delta tables via PyArrow datasets. DuckDB parses the Delta transaction log, ignores tombstoned parquet files, and executes multi-decade joins between solar flux and satellite decay counts in under 50 milliseconds.

7. Workflow Orchestration (Prefect Flow)

The platform coordinates all operational pipeline steps using **Prefect 3.8** (flows/orbitwatch_flow.py):

1. download_raw_data_task: Downloads fresh ephemerides and solar telemetry with automatic retries (retries=2, retry_delay=2s).
2. run_data_validation_task: Executes Great Expectations quality gates across all staging tables.
3. load_data_warehouse_task: Executes the atomic dimensional warehouse loader.

Failure Circuit-Breaker: The tasks are chained sequentially. In fault-injection tests (simulating an unreachable API), the download task exhausted its retries and transitioned to Failed, which immediately halted downstream validation and warehouse loading. This guarantees that corrupt upstream data never touches the warehouse.

8. Front-End: Operational Intelligence Dashboard

The front-end user interface is implemented as a single-page, responsive web application in **Streamlit** (app/dashboard.py):

- **Role-Based Security:** The application connects exclusively using the dashboard_user role. Permissions are strictly restricted to SELECT on schema dw, verified at runtime via SELECT current_user. Staging, raw, and audit modifications are blocked at the database level.
- **Executive KPI Cards:** Real-time counters displaying 70,953 catalogued satellites, 19,366 active GP snapshots, 3,333 solar weather observation months, and 35,677 atmospheric re-entry events.
- **Orbit Regime Population Analysis:** Interactive breakdown detailing satellite counts: LEO (62,290; 87.8%), MEO (3,688; 5.2%), UNKNOWN (2,052; 2.9%), HEO (1,986; 2.8%), and GEO (937; 1.3%).
- **Dual-Axis Solar vs. Decay Time Series:** Primary axis plots 10.7cm solar radio flux (sfu) and secondary axis plots monthly satellite re-entries over 70+ years.
- **Orbital Mechanics Snapshot Table:** Aggregates mean motion (rev/day), eccentricity, and inclination across orbital regimes.

9. Scientific Analysis: Solar Flux vs. Orbital Decay

A core research question of ORBITWATCH is quantifying the relationship between solar electromagnetic output and satellite re-entry rates:

Statistical Findings: Pearson correlation analysis across **808 monthly observations** reveals a correlation coefficient of **r = 0.7060**, indicating a *strong, statistically significant positive correlation* between solar radio flux (F10.7) and atmospheric decay events.

> ***Scientific Caveat (Correlation vs. Causation):*** *While solar EUV heating physically expands the thermosphere and increases aerodynamic drag, observed monthly decay totals are also modulated by human launch volume, mega-constellation end-of-life deorbiting policies, and historical space tracking coverage. Solar flux is the primary physical driver, but not the sole factor.*

10. Verification, Testing & Oral Defense Guide

The platform was verified through a comprehensive 43-test automated test suite (pytest -v) with 100% pass rate:

Test Suite	Scope / Functionality Verified	Tests	Outcome
test_cleaning.py	Domain cleaning, outlier detection, string stripping	4	PASSED
test_healthcheck.py	PostgreSQL, MongoDB, and Redpanda connectivity	3	PASSED
test_ingestion_api.py	FastAPI auth, duplicate keys, payload validation	5	PASSED
test_streaming.py	Deterministic UUID5 events, envelope parsing	3	PASSED
test_validation.py	Great Expectations Raw and Clean expectation suites	22	PASSED
test_warehouse.py	Date keys, stage reconciliation, zero orphan FKs, RBAC	6	PASSED

Key Oral Defense (Viva) Questions & Answers:

Q1: Why choose a Kimball Star Schema over a 3NF Core model?
A: Space domain analytics requires multi-dimensional slicing across time, regime, and solar flux. Star schemas denormalize dimension attributes, drastically reducing join overhead and producing predictable query plans for reporting tools.

Q2: How does Delta Lake prevent reading partial data during concurrent exports?
A: Delta Lake enforces ACID transactions via its JSON commit log (_delta_log). Readers only query files referenced in committed transaction versions; uncommitted or tombstoned parquet files are filtered out automatically.

Q3: What are the main limitations of the platform?
A: (1) Future epoch rule: GP data must satisfy EPOCH ≤ now(UTC), meaning forward-propagated ephemerides are quarantined. (2) K-index horizon: The SWPC 1-minute feed is limited to a rolling 24-hour window (359 rows), so it was excluded from multi-decade warehouse facts.